EXERCISE 09: K-MEANS CLUSTERING AND CLUSTER PREDICTION

College Data Science Laboratory Course — Practical Examination Study Guide
Source Code File: `e09_kmeans.py` | **Repository:** `https://github.com/akhilkvu6/ds` | **Algorithm Family:** Unsupervised Machine Learning — Partition-Based Clustering

1. Program Identification and Aim

- **File Name:** `e09_kmeans.py`
- **Academic Aim:** To implement the **K-Means Unsupervised Clustering Algorithm** from scratch using scikit-learn on synthetically generated 2-dimensional cluster data, partition $N=300$ unlabeled observations into $k=5$ natural clusters, identify and extract the converged geometric centroids, visualize the clustered data partitions with cluster color mapping alongside centroid positions, and predict the cluster membership for arbitrary new coordinate points.
- **Fundamental Paradigm Shift (CRITICAL EXAM POINT):**

  > **ONLY UNSUPERVISED LEARNING PROGRAM IN THE ENTIRE LAB CURRICULUM:**
  > Every other machine learning program in Exercises 05 to 10 is *Supervised* (trained using features X alongside ground truth target labels y). Exercise 09 is purely **Unsupervised**. The model receives strictly unlabeled spatial coordinates `data` (shape 300×2). There is no target vector y, no true category supervision, and no train/test data splitting. The algorithm autonomously discovers underlying structural groups purely based on geometric spatial proximity (Euclidean distance).

- **Core Output:** A Matplotlib scatter plot displaying 300 data points color-coded into 5 clusters, 5 red 'x' marks indicating centroid positions, 2 highlighted query points, and terminal output displaying predicted cluster assignments for 3 arbitrary test points.

2. Prerequisites and Conceptual Foundations

- **Theoretical Background:** Unsupervised Learning, Centroid-based Partitioning, Euclidean Metric, Within-Cluster Sum of Squares (WCSS / Inertia), Lloyd's Iterative Optimization Algorithm, K-Means++ Initialization Strategy.
- **Required Python Modules:**
  - `numpy`: Fundamental numerical array manipulation.
  - `sklearn.cluster.KMeans`: Scikit-learn's optimized Lloyd/Elkan K-Means clustering estimator.
  - `sklearn.datasets.make_blobs`: Synthetic isotropic Gaussian blob generator for clustering experiments.
  - `matplotlib.pyplot`: Visual plotting and 2D scatter rendering.
- **System/Environment Assumptions:** Python 3.8+ with `numpy`, `scikit-learn`, and `matplotlib`. No local CSV or TXT dependencies. Data is generated dynamically in memory.

3. Complete Exam-Ready Algorithm (START ... STOP Format)

ALGORITHM: K-Means Clustering and Prediction

Step 1: START
Step 2: [Import Required Modules]
Import `numpy` as `np`.
Import `KMeans` from `sklearn.cluster`.
Import `make_blobs` from `sklearn.datasets`.
Import `matplotlib.pyplot` as `plt`.
Step 3: [Generate Synthetic 2D Cluster Data]
Generate 300 synthetic two-dimensional coordinate samples using `make_blobs(n_samples=300)`.
Assign coordinate array to `data` and discard true label outputs using underscore `_` to maintain an unsupervised setting.
Step 4: [Configure and Train K-Means Model]
Initialize `KMeans(n_clusters=5, random_state=42, n_init="auto")`.
Fit the model to the data matrix: `kmeans.fit(data)`.
Step 5: [Extract Cluster Assignments and Centroids]
Retrieve the cluster assignment array (integers 0 to 4) from `kmeans.labels_`.
Retrieve the converged centroid coordinates (shape 5×2) from `kmeans.cluster_centers_`.
Step 6: [Plot Clustered Data Points]
Render scatter plot of `data[:, 0]` vs `data[:, 1]`.
Color data points by their assigned cluster indices using `c=labels` with colormap `cmap="viridis"`.
Step 7: [Overlay Cluster Centroids]
Render scatter plot of `centroids[:, 0]` vs `centroids[:, 1]` using red color and cross markers: `color="red", marker="x"`.
Step 8: [Add Legend and Highlight Test Points]
Attach legend showing unique cluster labels: `plt.legend(list(set(labels)))`.
Scatter highlight query points `[2, 6]` and `[-4, 6]` in red: `plt.scatter([2, 6], [-4, 6], color="red")`.
Step 9: [Display Plot Canvas]
Display interactive graphical figure window using `plt.show()`.
Step 10: [Predict Clusters for Unseen Query Points]
Execute `kmeans.predict([[2, -4], [6, 6], [6, -2]])` to compute nearest centroid indices for each query point.
Print the resulting cluster prediction array to standard output.
Step 11: STOP

4. Complete Theory, Formulas, and Numerical Worked Example

4.1 K-Means Objective Function: Within-Cluster Sum of Squares (Inertia)

Given a dataset of N points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \in \mathbb{R}^d$, K-Means partitions the points into k disjoint clusters $S = \{S_1, S_2, \dots, S_k\}$ to minimize the objective function (Inertia / WCSS):

$$J = \sum_{j=1}^k \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

Where $\boldsymbol{\mu}_j$ is the centroid (mean vector) of cluster S_j:

$$\boldsymbol{\mu}_j = \frac{1}{|S_j|} \sum_{\mathbf{x}_i \in S_j} \mathbf{x}_i$$

4.2 Lloyd's Alternating Optimization Algorithm

K-Means solves this non-convex NP-hard optimization problem using Lloyd's heuristic iterative procedure:

1. **Initialization:** Select k initial centroids $\boldsymbol{\mu}_1^{(0)}, \dots, \boldsymbol{\mu}_k^{(0)}$. Scikit-learn defaults to `init='k-means++'`, which seeds centroids probabilistically proportional to squared distance from existing seeds to prevent poor local optima.
2. **Assignment Step (Expectation):** Assign each sample point $\mathbf{x}_i$ to its nearest centroid according to Euclidean distance: $$S_j^{(t)} = \left\{ \mathbf{x}_i : \|\mathbf{x}_i - \boldsymbol{\mu}_j^{(t)}\|^2 \le \|\mathbf{x}_i - \boldsymbol{\mu}_m^{(t)}\|^2, \quad \forall m \in \{1, \dots, k\} \right\}$$
3. **Update Step (Maximization):** Recompute each centroid as the arithmetic mean of all points assigned to that cluster in step 2: $$\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{|S_j^{(t)}|} \sum_{\mathbf{x}_i \in S_j^{(t)}} \mathbf{x}_i$$
4. **Convergence Criterion:** Repeat Steps 2 and 3 until centroids stabilize ($\|\boldsymbol{\mu}_j^{(t+1)} - \boldsymbol{\mu}_j^{(t)}\| < \epsilon$) or maximum iterations (`max_iter=300`) are reached.

4.3 Step-by-Step Worked Numerical Example

Worked Example: Cluster 4 data points in 2D space into $k=2$ clusters.

Data Points: $P_1 = (1, 1)$, $P_2 = (2, 1)$, $P_3 = (4, 3)$, $P_4 = (5, 4)$
Initial Centroids: $\boldsymbol{\mu}_1 = (1, 1)$, $\boldsymbol{\mu}_2 = (5, 4)$

Iteration 1 — Assignment Step (Euclidean Distance $d = \sqrt{(x - \mu_x)^2 + (y - \mu_y)^2}$):
• For $P_1(1, 1)$: $d(P_1, \boldsymbol{\mu}_1) = 0$; $d(P_1, \boldsymbol{\mu}_2) = \sqrt{(1-5)^2 + (1-4)^2} = \sqrt{16+9} = 5.0$ → **Assign to Cluster 1**
• For $P_2(2, 1)$: $d(P_2, \boldsymbol{\mu}_1) = \sqrt{(2-1)^2 + (1-1)^2} = 1.0$; $d(P_2, \boldsymbol{\mu}_2) = \sqrt{(2-5)^2 + (1-4)^2} = \sqrt{9+9} = 4.24$ → **Assign to Cluster 1**
• For $P_3(4, 3)$: $d(P_3, \boldsymbol{\mu}_1) = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{9+4} = 3.61$; $d(P_3, \boldsymbol{\mu}_2) = \sqrt{(4-5)^2 + (3-4)^2} = \sqrt{1+1} = 1.41$ → **Assign to Cluster 2**
• For $P_4(5, 4)$: $d(P_4, \boldsymbol{\mu}_1) = \sqrt{(5-1)^2 + (4-1)^2} = 5.0$; $d(P_4, \boldsymbol{\mu}_2) = 0$ → **Assign to Cluster 2**

Resulting Partitions: $S_1 = \{P_1, P_2\}$, $S_2 = \{P_3, P_4\}$

Iteration 1 — Update Step (Recompute Centroids):
$$\boldsymbol{\mu}_1^{\text{new}} = \left(\frac{1 + 2}{2}, \frac{1 + 1}{2} \right) = (1.5, 1.0)$$ $$\boldsymbol{\mu}_2^{\text{new}} = \left(\frac{4 + 5}{2}, \frac{3 + 4}{2} \right) = (4.5, 3.5)$$
Iteration 2: Re-measuring distances confirms that assignments remain unchanged ($S_1 = \{P_1, P_2\}$, $S_2 = \{P_3, P_4\}$). Centroids have converged!

4.4 Choosing Optimal k: The Elbow Method and Silhouette Analysis

- **The Elbow Method:** Run K-Means for a range of values (e.g. $k = 1$ to 10) and compute the total within-cluster sum of squares (Inertia) for each k. Plotting Inertia against k produces an arm-like curve. The optimal k is chosen at the distinct "elbow" point, where adding another cluster yields diminishing returns in inertia reduction.
- **Silhouette Coefficient:** For sample i, $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$, where $a(i)$ is mean intra-cluster distance and $b(i)$ is mean nearest-cluster distance. Ranges from -1 (poor) to $+1$ (perfect separation).

5. Complete Original Source Code and Block-by-Block Walkthrough

5.1 Verbatim Source Code (`e09_kmeans.py`)

```
# Exercise 9: K-Means Clustering

# Question 1: Implement K-Means clustering using generated data and predict the clusters of new data points.

import numpy as np

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

data, _ = make_blobs(n_samples=300)

data

kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init="auto"
).fit(data)

labels = kmeans.labels_

set(labels)

import matplotlib.pyplot as plt

centroids = kmeans.cluster_centers_

plt.scatter(
    data[:, 0],
    data[:, 1],
    c=labels,
    cmap="viridis"
)

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    color="red",
    marker="x"
)

plt.legend(list(set(labels)))

plt.scatter(
    [2, 6],
    [-4, 6],
    color="red"
)

plt.show()

print(kmeans.predict([[2, -4], [6, 6], [6, -2]]))
```

5.2 Block-by-Block Walkthrough

Lines	Code Block	Detailed Execution Explanation
1–3	Header Comments	Identifies Exercise 9, K-Means Clustering, and the laboratory problem statement.
5–8	Module Imports	Imports `numpy`, `KMeans` from `sklearn.cluster`, and `make_blobs` from `sklearn.datasets`.
10	Synthetic Data Generation	Generates 300 isotropic Gaussian 2D points using `make_blobs(n_samples=300)`. Discards true cluster labels into `_`.
12	Standalone Expression	Jupyter/IPython artifact; outputs nothing in command-line Python execution. Harmless.
14–18	Model Definition & Fitting	Instantiates `KMeans` with $k=5$, `random_state=42`, and modern `n_init="auto"`, immediately fitting to `data` via method chaining `.fit(data)`.
20	Cluster Label Extraction	Extracts the array of cluster labels assigned to each sample: `kmeans.labels_` (shape `(300,)`, values $0, 1, 2, 3, 4$).
22	Unique Labels Expression	Notebook artifact evaluating unique cluster IDs: `set(labels)` yields `{0, 1, 2, 3, 4}`.
24	Late Import	Imports `matplotlib.pyplot as plt` right before visualization logic starts.
26	Centroid Extraction	Extracts cluster centroids coordinates: `kmeans.cluster_centers_` (shape `(5, 2)`).
28–33	Data Scatter Plot	Scatters 300 points in 2D space. `c=labels` colors each point by its assigned cluster; `cmap="viridis"` applies a sequential green-yellow-purple colormap.
35–40	Centroids Scatter Plot	Overlays the 5 centroids at `centroids[:, 0]` and `centroids[:, 1]` with red color and cross marker `marker="x"`.
42	Legend Rendering	Attaches legend using unique cluster labels `list(set(labels))`. (Note: harmless quirk explained in viva).
44–48	Query Points Scatter	Scatters two red dots at coordinates $(2, -4)$ and $(6, 6)$ on the plot canvas.
50	Figure Display	Renders and blocks until the graphical window is closed by the user.
52	Cluster Prediction	Predicts nearest cluster for three query points `[[2, -4], [6, 6], [6, -2]]` and prints array of cluster IDs.

6. Complete Function, Parameter, and Syntax Reference

6.1 Comprehensive 10-Point Technical Breakdown

1. `make_blobs(n_samples=300)` **:** Generates isotropic Gaussian blobs for clustering. Defaults: `n_features=2`, `centers=3` (unless modified), `cluster_std=1.0`. Returns tuple `(X, y)` where `_` discards `y`.
2. `data, _ = ...` **:** Tuple unpacking where underscore `_` is Python's conventional dummy variable for intentionally ignored values.
3. `KMeans(n_clusters=5, random_state=42, n_init="auto")` **:**
   - `n_clusters=5`: Number of clusters (k) and centroids to generate.
   - `random_state=42`: Fixes seed for random centroid seeding, ensuring reproducible clusters across runs.
   - `n_init="auto"`: Added in scikit-learn 1.2. Sets `n_init=1` when `init='k-means++'` is active, replacing legacy default of 10 runs to improve speed while suppressing deprecation warnings.
4. `.fit(data)` **:** Computes K-Means clustering on the input 2D array `data`. Modifies the estimator in place and returns the fitted instance.
5. `kmeans.labels_` **:** 1D ndarray of shape `(n_samples,)` containing integer labels from 0 to $k-1$ indicating which cluster each sample belongs to.
6. `kmeans.cluster_centers_` **:** 2D ndarray of shape `(n_clusters, n_features) = (5, 2)` containing the final (x, y) coordinates of all 5 cluster centroids.
7. `plt.scatter(x, y, c=labels, cmap="viridis")` **:** 2D scatter function. `c` maps scalar values to colormap colors. `cmap="viridis"` provides perceptually uniform colors (purple to yellow).
8. **Centroid Styling:** `color="red", marker="x"` sets cross markers in bold red to contrast sharply against circular data points.
9. `plt.scatter([2, 6], [-4, 6], color="red")` **:** Plots two individual points: $(2, -4)$ and $(6, 6)$ on the coordinate plane.
10. `kmeans.predict(X_new)` **:** Assigns new coordinate vectors to their closest centroid without refitting the model. Requires a 2D array of shape `(n_queries, 2)`.

6.2 Technical Coverage Checklist

Technical Concept / Item	Line Number	Status	Exam Relevance
Synthetic Cluster Generation	Line 10	✓ Verified	`make_blobs(n_samples=300)` with dummy label discard
KMeans Constructor Parameters	Lines 14–18	✓ Verified	`n_clusters=5`, `random_state=42`, `n_init="auto"`
Learned Model Attributes	Lines 20, 26	✓ Verified	Trailing underscore attributes: `labels_` and `cluster_centers_`
Colormap Scatter Plotting	Lines 28–33	✓ Verified	`c=labels, cmap="viridis"`
Centroid Visual Overlay	Lines 35–40	✓ Verified	`color="red", marker="x"`
Unseen Coordinate Prediction	Line 52	✓ Verified	`kmeans.predict([[2, -4], [6, 6], [6, -2]])`

7. Step-by-Step Execution Walkthrough and Variable State Trace

Step	Line	Action / Expression	Variable State / Output
1	10	`make_blobs(n_samples=300)`	`data.shape == (300, 2)`; `_` holds 300 ground-truth labels (ignored).
2	14–18	`KMeans(...).fit(data)`	`kmeans`: Fitted KMeans estimator with converged centroids.
3	20	Extract `labels_`	`labels.shape == (300,)`; array of integers $\in \{0, 1, 2, 3, 4\}$.
4	26	Extract `cluster_centers_`	`centroids.shape == (5, 2)`; 5 coordinate pairs.
5	28–33	Scatter data points	300 colored dots added to current Matplotlib axes.
6	35–40	Scatter centroids	5 red 'x' marks plotted at centroid coordinates.
7	44–48	Scatter query markers	Red dots plotted at $(2, -4)$ and $(6, 6)$.
8	50	`plt.show()`	GUI window displays plot. Execution blocks until window is closed.
9	52	`kmeans.predict(...)`	Terminal displays predicted cluster IDs: e.g. `[1 3 0]`.

8. Output and Practical Interpretation

8.1 Visual Graphical Output Description

> **Description of Matplotlib Graphical Window (Generated by `plt.show()`):**
> • **Data Distribution:** 300 circular dots distributed across 2D Euclidean coordinate space.
> • **Color Partitioning:** The dots are visually clustered into 5 distinct groups, shaded smoothly from deep purple to teal to bright yellow by the `viridis` colormap.
> • **Cluster Centroids:** Exactly 5 red `x` marks appear, each resting precisely at the geometric balance point (mean center) of its respective cluster group.
> • **Red Query Points:** Two solid red dots appear at coordinates $(2, -4)$ and $(6, 6)$, highlighting the locations of query samples relative to the existing cluster clouds.

8.2 Terminal Execution Output

```
[1 3 0]
```

Interpretation: The three test coordinates supplied to `predict()` are classified as follows:

- Point 1 $(2, -4)$ has minimum Euclidean distance to Centroid 1 → Assigned to **Cluster 1**.
- Point 2 $(6, 6)$ has minimum Euclidean distance to Centroid 3 → Assigned to **Cluster 3**.
- Point 3 $(6, -2)$ has minimum Euclidean distance to Centroid 0 → Assigned to **Cluster 0**.

Note: The exact cluster indices (0 to 4) assigned to these points may vary depending on the randomly generated blob cluster centers, but the points are always mapped to their mathematically closest centroid.

9. Exam Preparation: Lab Record Answer and Model Questions

9.1 Lab Record Standard Format (Write in 1 Page)

> **EXERCISE 9: K-MEANS CLUSTERING**
>
> **AIM:** To write a Python program implementing the K-Means unsupervised clustering algorithm on synthetic 2D data, visualize the clustered data points and centroids, and predict cluster memberships for new data points.
>
> **ALGORITHM:**
> 1. Import `numpy`, `KMeans` from `sklearn.cluster`, `make_blobs` from `sklearn.datasets`, and `matplotlib.pyplot`.
> 2. Generate 300 synthetic 2D observations using `make_blobs(n_samples=300)` and store in `data`.
> 3. Initialize `KMeans(n_clusters=5, random_state=42, n_init="auto")` and fit to `data`.
> 4. Extract cluster labels `kmeans.labels_` and centroid coordinates `kmeans.cluster_centers_`.
> 5. Plot data points using scatter plot with cluster coloring (`c=labels, cmap="viridis"`).
> 6. Overlay the 5 cluster centroids using red 'x' markers.
> 7. Display the graphical cluster plot using `plt.show()`.
> 8. Predict and print cluster assignments for unseen query points `[[2, -4], [6, 6], [6, -2]]`.
>
> **RESULT:** The K-Means clustering model was successfully built and executed. 300 samples were partitioned into 5 clusters, centroids were visualized, and query points were successfully assigned to clusters.

9.2 Common Exam Questions and Model Answers

- **2-Mark Question:** Differentiate between Supervised and Unsupervised Learning with reference to this lab.
  Answer: Supervised learning (e.g. KNN, Naive Bayes, Decision Trees) trains on input features X alongside known target labels y to predict future outputs. Unsupervised learning (K-Means) receives only features X without any labels y and discovers intrinsic groupings or patterns based on geometric proximity.
- **3-Mark Question:** What is the meaning of `n_init="auto"` in scikit-learn's `KMeans`?
  Answer: `n_init` specifies how many times the K-Means algorithm runs with different centroid seeds. In scikit-learn 1.2+, `n_init="auto"` dynamically sets the number of initializations to 1 when using the smart `k-means++` initialization strategy, or 10 when using random initialization, maximizing efficiency and eliminating deprecation warnings.
- **5-Mark Question:** Explain the working of Lloyd's algorithm for K-Means clustering.
  Answer: Lloyd's algorithm follows four sequential steps: (1) **Initialization:** Pick k initial centroids either randomly or using K-Means++. (2) **Assignment Step:** For every data point, compute Euclidean distance to all k centroids and assign each point to the closest centroid. (3) **Update Step:** Recompute each centroid's coordinates as the mathematical mean of all data points currently assigned to it: $\boldsymbol{\mu}_j = \frac{1}{|S_j|} \sum \mathbf{x}_i$. (4) **Convergence:** Repeat steps 2 and 3 until cluster assignments no longer change or centroid movement drops below a defined tolerance threshold ϵ.

C. Predict-the-Output Questions

1. **Question:** What are the dimensions of `kmeans.cluster_centers_` for 5 clusters on 2D data?
   Answer: Shape is `(5, 2)` (5 coordinates of (x, y) centroids).
2. **Question:** What is the shape of `kmeans.labels_` for 300 data points?
   Answer: Shape is `(300,)` with integer labels $\in \{0, 1, 2, 3, 4\}$.

D. Identify-the-Function Questions

1. **Question:** Which function in `sklearn.datasets` generates synthetic isotropic Gaussian blobs for clustering?
   Answer: `make_blobs()`.
2. **Question:** Which class in `sklearn.cluster` implements K-Means clustering?
   Answer: `KMeans()`.

3. **Question:** Which learned attribute contains the assigned cluster indices for each training point?
   Answer: `kmeans.labels_`.

10. Comprehensive Viva Voce Questions and Answers (16 Exam Questions)

Q1: What type of machine learning algorithm is K-Means?
Answer: It is an unsupervised, iterative, partition-based clustering algorithm.

Q2: Why is the target output discarded with underscore `_` in `data, _ = make_blobs()`?
Answer: `make_blobs()` returns both generated points and ground-truth cluster labels. In unsupervised clustering, models must not have access to ground-truth labels. The underscore `_` discards the labels to enforce a strictly unsupervised learning workflow.

Q3: How does K-Means define the distance between a point and a cluster centroid?
Answer: By default, it uses standard Euclidean distance: $d(\mathbf{x}, \boldsymbol{\mu}) = \sqrt{\sum_{i=1}^d (x_i - \mu_i)^2}$.

Q4: What is "Inertia" in K-Means clustering?
Answer: Inertia (or Within-Cluster Sum of Squares — WCSS) is the sum of squared Euclidean distances of samples to their closest cluster center. Lower inertia indicates tighter, more compact clusters.

Q5: What is the K-Means++ initialization algorithm and why is it preferred over random seeding?
Answer: Purely random centroid initialization can trap K-Means in poor local minima. K-Means++ spreads out initial centroids by choosing the first center randomly and subsequent centers with probability proportional to the squared distance from the nearest existing centroid.

Q6: What does the trailing underscore mean in `kmeans.labels_` and `kmeans.cluster_centers_`?
Answer: In scikit-learn conventions, an attribute name ending with an underscore indicates an estimated/learned parameter computed during the `fit()` process, rather than a user-configured hyperparameter.

Q7: What is the shape and content of `kmeans.cluster_centers_` in this program?
Answer: Shape is $(5, 2)$, representing the (x, y) coordinates of the 5 converged geometric centroids in 2-dimensional feature space.

Q8: What does `cmap="viridis"` do in `plt.scatter()`?
Answer: It specifies the colormap used to convert numeric values in `labels` (0 to 4) into distinct colors (blue, teal, green, yellow) for visual distinction of clusters.

Q9: How does `kmeans.predict()` assign new data points to clusters?
Answer: It calculates the Euclidean distance between each new query point and the 5 pre-computed centroids stored in `cluster_centers_`, returning the index of the closest centroid. It does not update the centroids.

Q10: What are the main limitations of K-Means clustering?
Answer: (1) Requires k to be specified in advance; (2) Assumes spherical, equally-sized clusters; (3) Highly sensitive to outliers; (4) Struggles with non-linear shapes (e.g. concentric circles or moons).

Q11: How do you determine the optimal number of clusters k in real-world data?
Answer: Using the **Elbow Method** (identifying where the rate of decrease in inertia slows down significantly) and the **Silhouette Score** (maximizing inter-cluster distance relative to intra-cluster distance).

Q12: Can K-Means converge to different solutions on different runs?
Answer: Yes, because the objective function is non-convex and can get stuck in local minima. Setting `random_state=42` ensures deterministic initial seeds and identical results every run.

Q13: Does K-Means require feature scaling in real-world datasets?
Answer: Yes! Because K-Means relies entirely on Euclidean distances, features with larger numerical ranges dominate distance calculations. Standardizing features using `StandardScaler` is essential when features have differing units.

Q14: Explain the quirk in `plt.legend(list(set(labels)))` in Line 42.
Answer: `plt.scatter` creates a single `PathCollection`. Passing a list of 5 labels to `plt.legend()` attaches only the first label to that single collection handle rather than labeling all 5 colors individually. However, Python executes the line without throwing an error.

Q15: What is the computational complexity of K-Means clustering?
Answer: $\mathcal{O}(I \cdot k \cdot N \cdot d)$, where I is number of iterations, k is number of clusters, N is number of data points, and d is number of dimensions (features).

Q16: How is K-Means different from K-Nearest Neighbors (KNN)?
Answer: K-Means is an **unsupervised clustering** algorithm that groups unlabeled points into k clusters. KNN is a **supervised classification** algorithm that predicts the label of an unknown point based on the k closest labeled training examples.

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. Why is K-Means called an "unsupervised" learning algorithm?
Answer: Because it groups data purely from feature geometry without target labels y or external ground-truth supervision.

2. What does `n_init="auto"` mean in modern scikit-learn?
Answer: It dynamically sets the number of centroid initializations to 1 when using `k-means++`, maximizing execution speed while avoiding deprecation warnings.

3. What are the two alternating steps in Lloyd's K-Means algorithm?
Answer: (1) Assignment Step: Assign each point to the closest centroid via Euclidean distance. (2) Update Step: Recompute each centroid as the mean of its assigned points.

4. What is Inertia (Within-Cluster Sum of Squares)?
Answer: The sum of squared Euclidean distances between each point and its closest cluster centroid: $\sum \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$.

5. Is the Elbow Method implemented in `e09_kmeans.py`?
Answer: No. The source code directly specifies $k=5$. The Elbow Method is additional theory explaining how to select optimal k by plotting inertia vs k.

11. Common Mistakes, Pitfalls, and Debugging Guide

Common Error / Pitfall	Root Cause	How to Fix / Debug
Confusing K-Means (k clusters) with KNN (k neighbors)	Both algorithms use the letter k and Euclidean distance, leading exam students to confuse them.	Remember: K-Means has NO labels y (Unsupervised Clustering). KNN has labels y (Supervised Classification).
`ValueError: Expected 2D array, got 1D array instead` in `predict()`	Passing a 1D coordinate pair like `kmeans.predict([2, -4])` instead of a 2D matrix.	Wrap in double brackets: `kmeans.predict([[2, -4]])` or pass multiple points: `[[2, -4], [6, 6], [6, -2]]`.
Missing trailing underscore on attributes	Writing `kmeans.labels` or `kmeans.cluster_centers` without trailing underscore.	Scikit-learn attributes learned during fitting always end with an underscore: `kmeans.labels_` and `kmeans.cluster_centers_`.
Passing labels to `fit()`	Writing `kmeans.fit(data, labels)` like in supervised models.	K-Means is unsupervised. Only pass features: `kmeans.fit(data)`.
Plot window blocks code execution	`plt.show()` pauses program execution until the window is manually closed.	This is expected behavior in Matplotlib. Close the GUI window to allow the terminal `print(kmeans.predict(...))` line to execute.

12. Memory Tricks, Mnemonics, and Last-Minute Rapid Revision

12.1 Six-Step Coding Flow Mnemonic: "B-M-F-L-C-P"

- **B — Blobs:** `data, _ = make_blobs(n_samples=300)`
- **M — Model:** `kmeans = KMeans(n_clusters=5, random_state=42, n_init="auto")`
- **F — Fit:** `.fit(data)`
- **L — Labels:** `labels = kmeans.labels_`
- **C — Centroids:** `centroids = kmeans.cluster_centers_`
- **P — Plot & Predict:** `plt.scatter(...)` → `plt.show()` → `kmeans.predict([[2, -4], ...])`

12.2 Quick Comparison: KNN vs K-Means

Property	KNN (Exercise 05)	K-Means (Exercise 09)
Learning Type	Supervised Classification	Unsupervised Clustering
Target y Required?	Yes (Ground truth labels)	No (Unlabeled spatial data)
Meaning of k	Number of nearest neighbors consulted	Number of cluster groups and centroids created
Training Phase	Lazy (Memorizes training instances)	Eager (Iteratively updates centroids until convergence)

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"` , `max_depth=4` , `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5` , `random_state=42` , `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line